TEST MODE

S

Stripe Payment Project

Complete Documentation — Version 2.0

Platform	Flask + Stripe Connect
Language	Python 3 / HTML / JS
Currency	INR (India Sandbox)
Port	localhost:4242
Date	March 2026

My SaaS App • Flask + Stripe Connect • Sandbox Environment

Contents

1 Project Overview **2**
- 1.1 Money Flow Diagram . . . 2
- 1.2 Fee Breakdown . . . 2

2 Project Architecture **3**
- 2.1 Architecture Diagram . . . 3
- 2.2 Folder Structure . . . 3

3 File-by-File Explanation **4**
- 3.1 payment.py — Main Entry Point . . . 4
- 3.2 backend/checkout.py — Create Payment Intent . . . 4
- 3.3 backend/confirm.py — Verify Payment . . . 6
- 3.4 backend/webhook.py — Event Listener . . . 6
- 3.5 backend/coupons.py — Coupon Validation . . . 7

4 Step-by-Step Process Flow **9**
- 4.1 Flow Diagram . . . 9
- 4.2 Detailed Steps . . . 9

5 Coupon / Redeem Code System **11**
- 5.1 Does Stripe Test Mode Provide Default Redeem Codes? . . . 11
- 5.2 Available Coupon Codes . . . 12
- 5.3 How Coupon Math Works . . . 12
- 5.4 Adding a New Coupon . . . 12
- 5.5 Coupon Validation Flow . . . 12

6 Stripe Connect — Receiver Setup **14**
- 6.1 What is Stripe Connect? . . . 14
- 6.2 Setup Steps . . . 14

7 Test Credentials & Test Cards **15**
- 7.1 Other Test Cards . . . 15

8 API Routes Reference **16**
- 8.1 Request / Response Examples . . . 16
  - 8.1.1 POST /validate-coupon . . . 16
  - 8.1.2 POST /create-payment-intent . . . 16
  - 8.1.3 POST /confirm-payment . . . 17

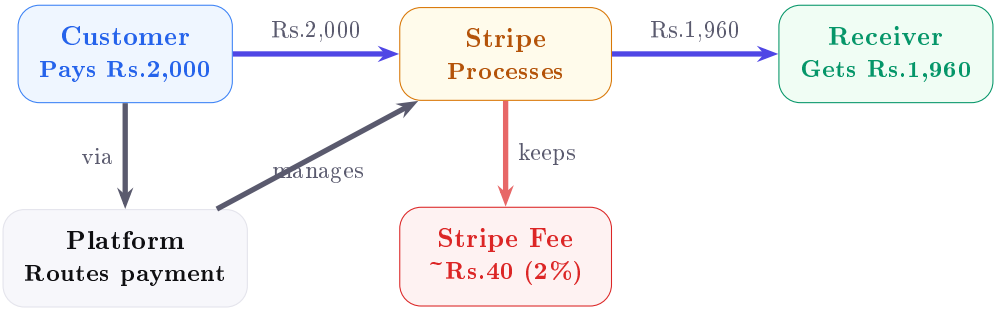
9 Mind Map Summary **18**

10 Troubleshooting Guide **19**
- 10.1 Where to See Transactions . . . 19
- 10.2 Installation & Running . . . 20

1. Project Overview

This project implements a complete payment platform using **Flask** and **Stripe Connect**. Customers pay via a checkout UI; Stripe deducts its processing fee, and the remainder is automatically routed to the designated receiver account. The platform acts as a marketplace intermediary — similar to Swiggy, Uber, or Fiverr.

1.1 Money Flow Diagram

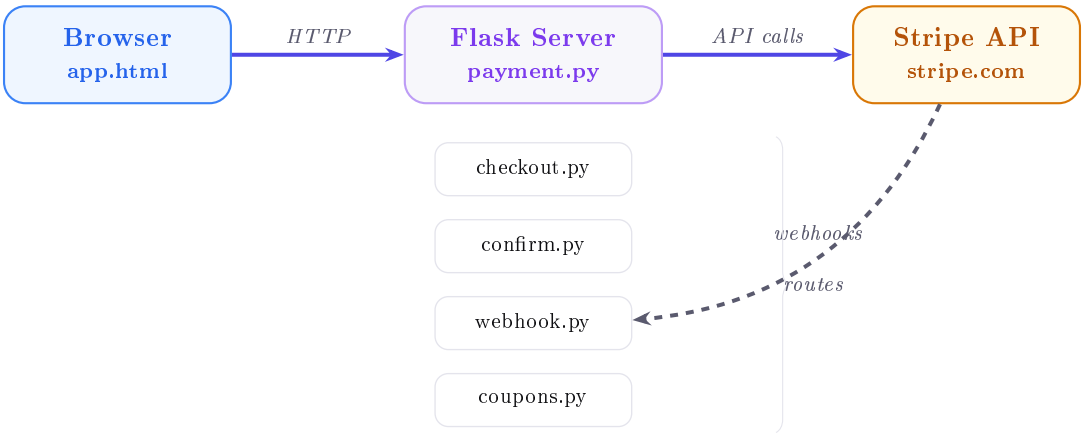


1.2 Fee Breakdown

Item	Amount (INR)	Description
Customer pays	Rs.2,000	Total charged to card
Stripe fee	Rs.40	~2% processing fee
Receiver gets	Rs.1,960	Net payout after Stripe fee
Platform fee	Rs.0	No platform cut (configurable)

2. Project Architecture

2.1 Architecture Diagram



2.2 Folder Structure

```
stripe-app/
|-- payment.py              # Main entry point (Flask app + server)
|-- requirements.txt        # Python dependencies
|-- backend/
|    |-- __init__.py
|    |-- checkout.py        # POST /create-payment-intent
|    |-- confirm.py         # POST /confirm-payment
|    |-- coupons.py         # Coupon validation logic (no Stripe
   needed)
|    `-- webhook.py         # POST /webhook  (Stripe events)
`-- frontend/
     |-- app.html           # Single-page checkout UI
     |-- css/
     |    `-- app.css       # Styles (Sora + JetBrains Mono)
     `-- js/
          `-- app.js        # Frontend logic (amount, coupon,
               Stripe.js)
```

3. File-by-File Explanation

3.1 payment.py — Main Entry Point

Role: Main switchboard. All routes register here. Server starts here. The `/config` endpoint must return *only* the publishable key — never the secret key.

```python
import stripe
from flask import Flask, jsonify, send_from_directory
from flask_cors import CORS
from backend.checkout import checkout_bp
from backend.confirm  import confirm_bp
from backend.webhook  import webhook_bp

app = Flask(__name__, static_folder="frontend")
CORS(app)

# YOUR STRIPE KEYS (India Sandbox account)
stripe.api_key  = "sk_test_51TCMU9B1f1mTeyhF..."    # Secret key (server only)
PUBLISHABLE_KEY = "pk_test_51TCMU9B1f1mTeyhF..."    # Publishable key (frontend)
WEBHOOK_SECRET  = "whsec_YOUR_WEBHOOK_SECRET_HERE"

app.config["PUBLISHABLE_KEY"] = PUBLISHABLE_KEY
app.config["WEBHOOK_SECRET"]  = WEBHOOK_SECRET

app.register_blueprint(checkout_bp)
app.register_blueprint(confirm_bp)
app.register_blueprint(webhook_bp)

@app.route("/")
def index():
    return send_from_directory("frontend", "app.html")

@app.route("/config")
def config():
    # IMPORTANT: return PUBLISHABLE_KEY only -- never stripe.api_key!
    return jsonify(publishableKey=app.config["PUBLISHABLE_KEY"])

if __name__ == "__main__":
    print("\n  Server running  ->  http://localhost:4242\n")
    app.run(port=4242, debug=True)
```

If `/config` returns `stripe.api_key` instead of `app.config["PUBLISHABLE_KEY"]`, the secret key becomes visible in the browser and the Stripe payment form renders as a blank purple box. Always return `pk_test_` only.

3.2 backend/checkout.py — Create Payment Intent

Role: When the customer clicks Pay, this creates a Stripe PaymentIntent. Applies coupon discount before creating the intent. Configures Stripe Connect transfer to receiver.

```python
import stripe
from flask import request, jsonify, Blueprint
from backend.coupons import validate_coupon

checkout_bp = Blueprint("checkout", __name__)
FIXED_RECEIVER_ACCOUNT = "acct_XXXXXXXXXX"  # Replace with receiver acct ID

@checkout_bp.route("/create-payment-intent", methods=["POST"])
def create_payment_intent():
    data          = request.json
    original_amt  = int(data.get("amount_inr", 100))
    email         = data.get("email", "")
    coupon_code   = data.get("coupon_code", "").strip().upper()

    redeemed_rupees = 0
    pay_rupees      = original_amt
    redeem_pct      = 0
    is_free         = False

    # Apply coupon if provided
    if coupon_code:
        result = validate_coupon(coupon_code, original_amt)
        if result["valid"]:
            redeemed_rupees = result["redeemed_rupees"]
            pay_rupees      = result["pay_rupees"]
            redeem_pct      = result["redeem_pct"]
            is_free         = result["is_free"]

    # Handle 100% free (FREE100 coupon)
    if is_free:
        return jsonify(isFree=True, payRupees=0,
                       redeemedRupees=redeemed_rupees,
                       originalRupees=original_amt, redeem_pct=redeem_pct)

    amount_paise = pay_rupees * 100     # INR -> paise (multiply by 100)

    customer = stripe.Customer.create(email=email) if email else None

    intent_params = dict(
        amount   = amount_paise,
        currency = "inr",               # India account -> INR works correctly
        customer = customer.id if customer else None,
        automatic_payment_methods = {"enabled": True},
        metadata = {
            "email": email, "original_rupees": original_amt,
            "redeemed_rupees": redeemed_rupees, "pay_rupees": pay_rupees,
            "redeem_pct": redeem_pct, "coupon_code": coupon_code,
        },
    )

    # Stripe Connect: route money to receiver account
```

```
    if FIXED_RECEIVER_ACCOUNT and not FIXED_RECEIVER_ACCOUNT.startswith("acct_XXX"):
        intent_params["transfer_data"] = {"destination": FIXED_RECEIVER_ACCOUNT}

    intent = stripe.PaymentIntent.create(**intent_params)
    return jsonify(clientSecret=intent.client_secret,
                   paymentIntentId=intent.id, payRupees=pay_rupees,
                   redeemedRupees=redeemed_rupees, isFree=False)
```

3.3 backend/confirm.py — Verify Payment

Role: After payment, verify with Stripe that the payment actually succeeded before showing the success screen.

```
import stripe
from flask import request, jsonify, Blueprint

confirm_bp = Blueprint("confirm", __name__)

@confirm_bp.route("/confirm-payment", methods=["POST"])
def confirm_payment():
    data              = request.json
    payment_intent_id = data.get("paymentIntentId")

    if not payment_intent_id:
        return jsonify(error="paymentIntentId is required"), 400

    intent = stripe.PaymentIntent.retrieve(payment_intent_id)

    if intent.status == "succeeded":
        meta = intent.metadata
        pay_rupees = intent.amount // 100    # paise -> rupees
        return jsonify(
            success        = True,
            status         = intent.status,
            payRupees      = pay_rupees,
            originalRupees = int(meta.get("original_rupees", pay_rupees)),
            redeemedRupees = int(meta.get("redeemed_rupees", 0)),
            redeemPct      = int(meta.get("redeem_pct", 0)),
            couponCode     = meta.get("coupon_code", ""),
            message        = "Payment successful!",
        )
    else:
        return jsonify(success=False, status=intent.status,
                       message=f"Payment status: {intent.status}"), 400
```

3.4 backend/webhook.py — Event Listener

Role: Stripe sends real-time events here. Always verify the `Stripe-Signature` header before processing.

```python
import stripe
from flask import request, jsonify, Blueprint, current_app

webhook_bp = Blueprint("webhook", __name__)

@webhook_bp.route("/webhook", methods=["POST"])
def webhook():
    payload    = request.data
    sig_header = request.headers.get("Stripe-Signature")
    secret     = current_app.config.get("WEBHOOK_SECRET", "")

    try:
        event = stripe.Webhook.construct_event(payload, sig_header,
            secret)
    except ValueError:
        return jsonify(error="Invalid payload"), 400
    except stripe.error.SignatureVerificationError:
        return jsonify(error="Invalid signature"), 400

    obj = event["data"]["object"]

    if   event["type"] == "payment_intent.succeeded":
        _handle_succeeded(obj)
    elif event["type"] == "payment_intent.payment_failed":
        _handle_failed(obj)
    elif event["type"] == "transfer.created":
        _handle_transfer(obj)
    elif event["type"] == "charge.refunded":
        _handle_refund(obj)

    return jsonify(received=True), 200

def _handle_succeeded(pi):
    meta = pi.get("metadata", {})
    print(f"  SUCCEEDED  | {pi['id']} | Rs.{pi['amount']//100:,}")
    print(f"  Coupon: {meta.get('coupon_code','none')} | "
          f"Redeemed: Rs.{meta.get('redeemed_rupees',0)}")

def _handle_failed(pi):
    err = (pi.get("last_payment_error") or
        {}).get("message","Unknown")
    print(f"  FAILED  | {pi['id']} | {err}")

def _handle_transfer(t):
    print(f"  TRANSFER  | Rs.{t['amount']//100:,} ->
        {t.get('destination')}")

def _handle_refund(ch):
    print(f"  REFUND  | Rs.{ch['amount_refunded']//100:,}")
```

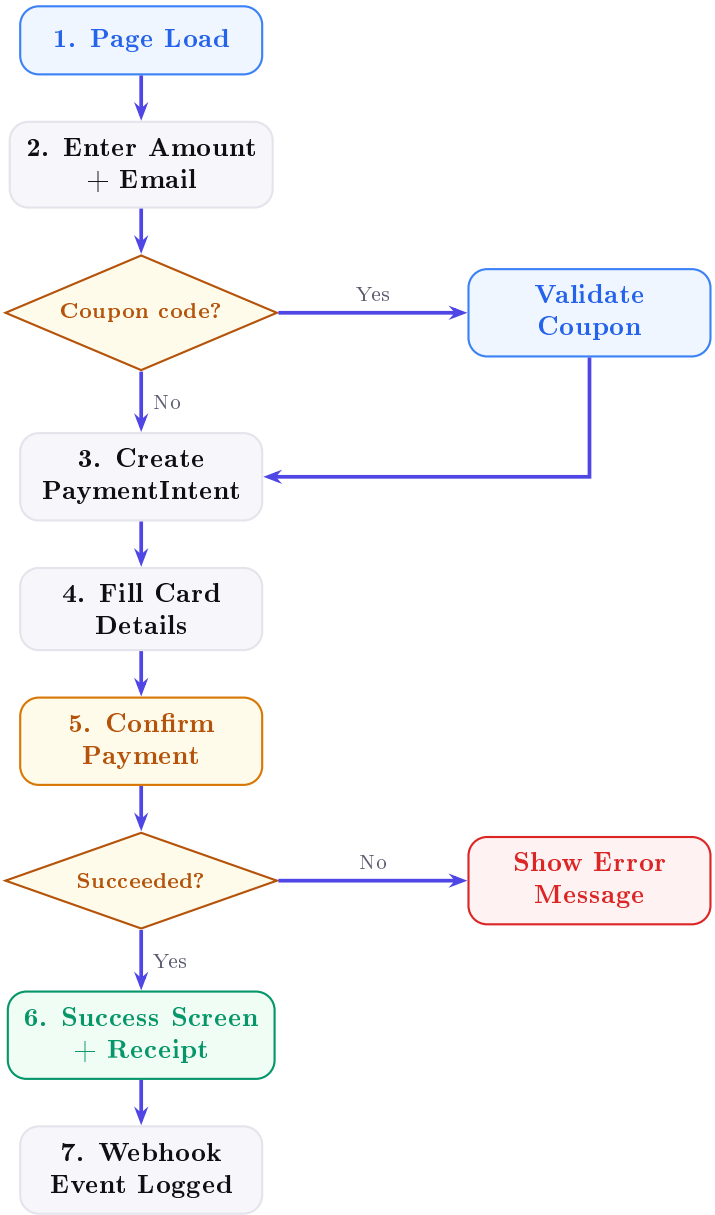
3.5 backend/coupons.py — Coupon Validation

Role: All coupon logic lives here. No Stripe Coupon API is used — coupons are defined as a plain Python dictionary.

```python
COUPONS = {
    "REDEEM10":  {"redeem_pct": 10,  "description": "10% redeemed -- pay 90%"},
    "REDEEM20":  {"redeem_pct": 20,  "description": "20% redeemed -- pay 80%"},
    "REDEEM30":  {"redeem_pct": 30,  "description": "30% redeemed -- pay 70%"},
    "HALF50":    {"redeem_pct": 50,  "description": "50% redeemed -- pay 50%"},
    "REDEEM70":  {"redeem_pct": 70,  "description": "70% redeemed -- pay 30%"},
    "FREE100":   {"redeem_pct": 100, "description": "100% redeemed -- FREE"},
    "WELCOME15": {"redeem_pct": 15,  "description": "Welcome -- 15% redeemed"},
}

def validate_coupon(code: str, original_amount: int) -> dict:
    code   = code.strip().upper()
    coupon = COUPONS.get(code)

    if not coupon:
        return {"valid": False, "error": f'"{code}" is not a valid redeem code'}

    redeem_pct      = coupon["redeem_pct"]
    redeemed_rupees = int(original_amount * redeem_pct / 100)
    pay_rupees      = original_amount - redeemed_rupees

    return {
        "valid":           True,
        "code":            code,
        "redeem_pct":      redeem_pct,
        "pay_pct":         100 - redeem_pct,
        "redeemed_rupees": redeemed_rupees,
        "pay_rupees":      pay_rupees,
        "is_free":         redeem_pct == 100,
        "description":     coupon["description"],
        "original_amount": original_amount,
    }
```

4. Step-by-Step Process Flow

4.1 Flow Diagram



4.2 Detailed Steps

1 Page Load

Customer opens `localhost:4242`. `app.html` loads. `/config` is called — returns the publishable key (`pk_test_...`). Stripe.js initialises.

2 Enter Amount + Email

Customer types amount or clicks a quick-select chip (Rs.100, Rs.500, Rs.1,000, Rs.2,000, Rs.5,000). Enters email address.

3 Apply Coupon (Optional)

Customer enters redeem code (e.g. `HALF50`). `/validate-coupon` is called. If valid, the discounted amount is shown in the summary strip.

4 Create PaymentIntent

On clicking Proceed to Pay, `/create-payment-intent` is called. Stripe creates a PaymentIntent with the final amount in INR (paise). A `clientSecret` is returned.

5 Fill Card Details

The Stripe Payment Element renders using the `clientSecret`. Customer fills in card details. Test card: `4242 4242 4242 4242`.

6 Confirm Payment

`stripe.confirmPayment()` is called. Stripe processes the card. `/confirm-payment` verifies the status.

7 Success Screen

Receipt shows: original amount, coupon savings, amount paid, Stripe fee, and receiver payout.

8 Webhook Logged

Stripe sends `payment_intent.succeeded` to `/webhook`. All details are logged server-side.

5. Coupon / Redeem Code System

5.1 Does Stripe Test Mode Provide Default Redeem Codes?

Stripe does NOT provide any default redeem codes in Test Mode. When you create a new Stripe Sandbox account, there are **zero** pre-built coupon codes waiting for you. The Stripe dashboard has a "Coupons" section (Billing → Coupons), but it is completely empty by default — Stripe never auto-creates any codes.

Also No — and this is the key point for this project.
Stripe's built-in Coupon API is meant for *subscription billing* (recurring charges). **This project does not use Stripe's Coupon API at all.** Instead, all redeem codes are defined as a plain Python dictionary inside `backend/coupons.py`. The validation happens entirely on our server — Stripe never sees the coupon code.

Summary:

- Stripe Test Mode default codes? **None exist.**
- Do we use Stripe's Coupon/Promotion API? **No.**
- Where are our codes defined? **In `backend/coupons.py` only.**
- Do we need to configure anything in Stripe dashboard for coupons? **No.**

Our coupon system is 100% custom and self-contained:

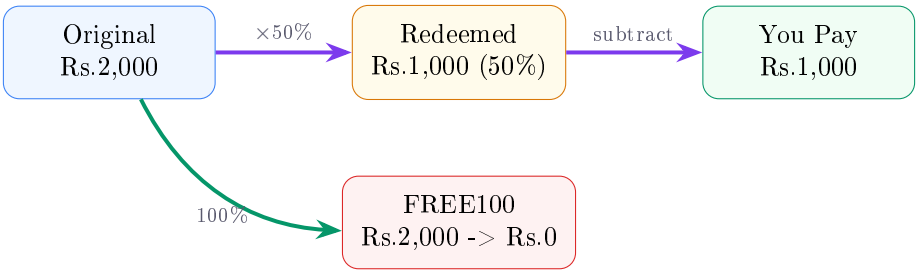
1. Customer enters a redeem code (e.g. `HALF50`) in the frontend.
2. Frontend calls `POST /validate-coupon` on our Flask server.
3. `validate_coupon()` looks up the code in the `COUPONS` Python dict.
4. If found: calculates discounted amount and returns it to frontend.
5. Discounted amount is used when creating the Stripe PaymentIntent.
6. Stripe only ever sees the *final discounted amount* — it has no knowledge of the coupon.

5.2 Available Coupon Codes

Code	Redeem %	Pay %	Description
REDEEM10	10%	90%	10% redeemed — customer pays 90%
REDEEM20	20%	80%	20% redeemed — customer pays 80%
REDEEM30	30%	70%	30% redeemed — customer pays 70%
HALF50	50%	50%	Half and half — pay only 50%
REDEEM70	70%	30%	70% redeemed — pay only 30%
FREE100	100%	0%	Completely FREE — no payment needed
WELCOME15	15%	85%	Welcome offer — 15% off

5.3 How Coupon Math Works

Given a coupon with `redeem_pct = 50` and original amount = Rs.2,000:



5.4 Adding a New Coupon

Simply add an entry to the `COUPONS` dict in `backend/coupons.py` — no Stripe dashboard changes needed:

```
# Add inside COUPONS dict in backend/coupons.py
"SUMMER25": {
    "redeem_pct":  25,
    "description": "Summer sale -- 25% redeemed",
},
"LOYALTY40": {
    "redeem_pct":  40,
    "description": "Loyalty reward -- 40% redeemed",
},
# Restart server -> ready to use immediately!
```

5.5 Coupon Validation Flow

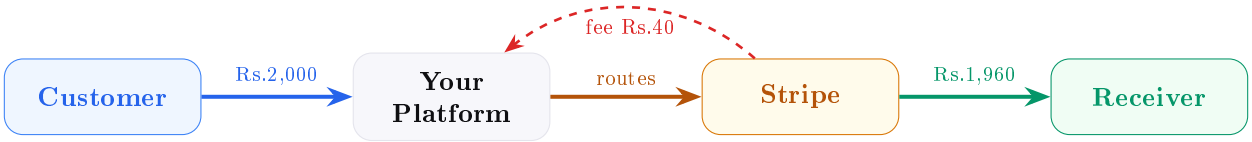
1. Customer enters code (e.g. `half50` or `HALF50`) — auto-uppercased.

2. Frontend calls `POST /validate-coupon` with `{code, amount_inr}`.
3. `validate_coupon()` checks if code exists in `COUPONS` dict.
4. If valid: returns `redeemed_rupees`, `pay_rupees`, `redeem_pct`.
5. If invalid: returns `{valid: false, error: "..."}`.
6. Frontend shows green success strip with discount summary.
7. On proceed: `/create-payment-intent` uses `pay_rupees` (discounted amount).
8. For `FREE100`: Stripe PaymentIntent is never created — success shown immediately.

6. Stripe Connect — Receiver Setup

6.1 What is Stripe Connect?

Stripe Connect allows you to route payments to third-party receiver accounts. The platform acts as the intermediary.



Real-world examples: Swiggy (restaurant gets money), Uber (driver gets money), Fiverr (freelancer gets money).

6.2 Setup Steps

1 Enable Connect

Sandbox Dashboard → Connect → Get started → Build a marketplace → Express accounts.

2 Add Receiver

Open `localhost:4242` → Add New Receiver → Enter Name + Email → Create & Onboard.

3 Complete Onboarding

A new tab opens with Stripe's onboarding form. Fill in test details and submit.

4 Copy Account ID

After onboarding: Connect → Accounts → click account → copy the `acct_XXXX` ID.

5 Set in Code

Paste `acct_XXXX` into `FIXED_RECEIVER_ACCOUNT` in `checkout.py`.

7. Test Credentials & Test Cards

These credentials work **only** in Stripe Test Mode / Sandbox. No real money is charged.

Field	Value	Note
Card Number	`4242 4242 4242 4242`	Main success card
Expiry Date	`12/29`	Any future date works
CVC	`123`	Any 3 digits work
ZIP Code	`42424`	Any 5 digits work
Email	`customer@test.com`	Any valid email

7.1 Other Test Cards

Card Number	Result	Use Case
`4242 4242 4242 4242`	**Success**	Normal payment
`4000 0000 0000 0002`	**Declined**	Test failure handling
`4000 0025 0000 3155`	**3D Secure**	Extra auth required
`4000 0000 0000 9995`	**Insuff. funds**	Test error messages
`4000 0000 0000 0069`	**Expired card**	Expired card scenario

8. API Routes Reference

Route	Method	Description
/	GET	Serve app.html
/config	GET	Return publishable key (pk_test_)
/validate-coupon	POST	Validate coupon code + calculate discount
/create-payment-intent	POST	Create Stripe PaymentIntent (INR)
/confirm-payment	POST	Verify payment status with Stripe
/accounts	GET	List connected receiver accounts
/create-account	POST	Onboard new receiver via Connect
/webhook	POST	Receive Stripe events (signature verified)
/onboard-complete	GET	Onboarding success redirect page
/onboard-refresh	GET	Onboarding expired redirect page

8.1 Request / Response Examples

POST /validate-coupon

```
# Request
{"code": "HALF50", "amount_inr": 2000}

# Response (valid)
{"valid": true, "code": "HALF50", "redeem_pct": 50,
 "redeemed_rupees": 1000, "pay_rupees": 1000, "is_free": false}

# Response (invalid)
{"valid": false, "error": "\"BADCODE\" is not a valid redeem code"}
```

POST /create-payment-intent

```
# Request
{"amount_inr": 2000, "email": "customer@test.com", "coupon_code":
   "HALF50"}

# Response
{"clientSecret": "pi_XXXX_secret_XXXX", "paymentIntentId": "pi_XXXX",
 "payRupees": 1000, "redeemedRupees": 1000, "isFree": false}
```

POST /confirm-payment

```
# Request
{"paymentIntentId": "pi_XXXXXXXXXX"}

# Response (success)
{"success": true, "status": "succeeded", "payRupees": 1000,
 "originalRupees": 2000, "redeemedRupees": 1000,
 "redeemPct": 50, "couponCode": "HALF50"}
```

9. Mind Map Summary

10. Troubleshooting Guide

Problem	Fix
Transactions show as USD / Incomplete	Switch to the new India Sandbox account keys (`sk_test_51TCMU9B1f1mTeyhF...`). Country must be India for INR to work.
Payment form shows `sk_test_` / purple blank box	`/config` is returning `stripe.api_key` (secret key). Fix: `return jsonify(publishableKey=app.config["PUBLISHABLE_KEY"])`. Only `pk_test_` should reach the browser.
Cannot connect to backend	Run `python payment.py` and ensure the server is on port 4242.
No connected accounts error	Stripe Sandbox → Connect → Get started → Marketplace → Express accounts.
Coupon code not working	Coupons are in `backend/coupons.py` — not in Stripe dashboard. Check the code exists in the `COUPONS` dict. Codes are auto-uppercased.
Payment failed with test card	Use `4242 4242 4242 4242`, any future expiry, any 3-digit CVC.
Webhook signature error	Run `stripe listen -forward-to localhost:4242/webhook` and copy the `whsec_...` secret into `WEBHOOK_SECRET`.
pip install fails	Use `pip install -r requirements.txt` (the `-r` flag is mandatory).
FREE100 coupon creates PaymentIntent	Check `is_free` flag in `/create-payment-intent`: if `is_free=True`, return early without calling `stripe.PaymentIntent.create()`.
Receiver not receiving funds	Ensure `FIXED_RECEIVER_ACCOUNT` in `checkout.py` is set to the correct `acct_XXXX` ID.

10.1 Where to See Transactions

What to Check	Dashboard URL
All payments	`dashboard.stripe.com/test/payments`
Balance	`dashboard.stripe.com/test/balance`
Connected accounts	`dashboard.stripe.com/test/connect/accounts`
Webhook logs	`dashboard.stripe.com/test/webhooks`

10.2 Installation & Running

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run the server
python payment.py

# 3. Open in browser
#    http://localhost:4242

# 4. For webhooks (optional but recommended)
stripe listen --forward-to localhost:4242/webhook
```

requirements.txt:

```
flask==3.0.3
stripe==10.5.0
flask-cors==4.0.1
```

Stripe Payment Project · Flask + Stripe Connect · India Sandbox · v2.0